Java Core
Interview Preparation Guide

A practical playbook for developers with 2-3 years of experience

Compiled by Surya Mahesh Kolisetty

How to Use This Guide

I put this guide together after sitting on both sides of the interview table for years, first as a nervous candidate and later as the person asking the questions. What struck me over time is how predictable Java interviews really are. The same handful of topics come up again and again, while entire chapters of the textbooks I once memorised almost never get mentioned.

So this is not a textbook. It is arranged the way interviews actually unfold. The topics that show up in nearly every conversation sit right at the front. The rare, trivia-style questions are pushed to the back where they belong. If you have only a weekend before an interview, start at Part 1 and stop wherever you run out of time. You will still have covered the things that matter most.

Every question carries a short priority tag so you can judge, at a glance, how much attention it deserves. Read the answers out loud if you can. Interviews are spoken conversations, and the phrasing you rehearse here is the phrasing that will come out under pressure.

Priority Labels

Priority	What It Means	How Often It Comes Up
Must Know	Expected in almost every interview	75% or more
High Priority	Comes up often	50 to 74%
Medium Priority	Occasional, worth a review	25 to 49%
Good to Know	Rare, read only if you have time	Under 25%

Part 1 - Java Collections

Java Collections is the single most asked topic in Java interviews. Expect at least 3-5 questions from this area in every technical round. HashMap internal working alone can take up 15 minutes of an interview.

MUST KNOW

HashMap Internal Working

HashMap is the most frequently asked data structure topic in Java interviews. Interviewers probe candidates on hashing, collision resolution, resizing, and the transition from linked list to tree structure introduced in Java 8.

How HashMap Works Internally

A HashMap stores key-value pairs in an array of buckets (called table). When you call put(key, value):

1. The hashCode() of the key is computed

2. The hash is spread using an internal perturbation function: hash = key.hashCode() ^ (key.hashCode() >>> 16)

3. The bucket index is calculated: index = hash & (capacity - 1)

4. If the bucket is empty, a new Node is placed there

5. If the bucket is occupied (collision), the new entry is appended to the linked list at that bucket

6. From Java 8: if a bucket's linked list grows beyond 8 nodes (and capacity >= 64), it converts to a balanced red-black tree

```
// Simplified internal structure of HashMap
public class HashMap<K, V> {
    static final int DEFAULT_INITIAL_CAPACITY = 16;
    static final float DEFAULT_LOAD_FACTOR = 0.75f;
    static final int TREEIFY_THRESHOLD = 8;

    transient Node<K, V>[] table;
    int threshold; // capacity * loadFactor

    static class Node<K, V> {
        final int hash;
        final K key;
        V value;
        Node<K, V> next;
    }
}
```

Resizing (Rehashing)

When the number of entries exceeds capacity * loadFactor (default: 16 * 0.75 = 12), HashMap doubles its capacity and rehashes all entries. This is an O(n) operation and is the primary reason HashMap can have worst-case performance spikes.

```
// Demonstrating resize behavior
Map<String, Integer> map = new HashMap<>(4, 0.75f);
// Threshold = 4 * 0.75 = 3
map.put("A", 1); // size=1, no resize
map.put("B", 2); // size=2, no resize
map.put("C", 3); // size=3, no resize
map.put("D", 4); // size=4 > threshold=3, RESIZE to capacity 8
```

MUST KNOW

What is the time complexity of HashMap get() and put() operations?

Typically asked as: Most Frequently Asked

Average case is O(1) for both get() and put(). However, the worst case depends on the Java version:

- Before Java 8: O(n) when all keys hash to the same bucket, forming a long linked list
- Java 8+: O(log n) in the worst case because long chains (>8 nodes) are converted to red-black trees

The O(1) average assumes a good hash function that distributes keys uniformly across buckets. A poor hashCode() implementation that returns the same value for all objects degrades HashMap to a linked list.

MUST KNOW

What happens when two keys have the same hashCode in a HashMap?

Typically asked as: Most Frequently Asked

This is called a hash collision. When two keys produce the same bucket index:

1. Both entries are stored in the same bucket

2. They form a linked list (or tree if chain length > 8)

3. During get(), HashMap first finds the bucket, then iterates through the chain calling equals() on each key to find the exact match

This is why both hashCode() and equals() contracts must be properly implemented - hashCode() determines the bucket, equals() determines the exact key match within that bucket.

```
public class Employee {
    private int id;
    private String name;

    @Override
    public int hashCode() {
        return Objects.hash(id, name);
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Employee)) return false;
        Employee e = (Employee) o;
        return id == e.id && Objects.equals(name, e.name);
    }
}
```

HIGH PRIORITY

Why is the initial capacity of HashMap a power of 2?

Typically asked as: Follow-up

HashMap uses bitwise AND (hash & (capacity - 1)) instead of modulo (hash % capacity) to calculate the bucket index. This only works correctly when capacity is a power of 2, because (2^n - 1) produces a bitmask of all 1s. This is faster than modulo division and distributes entries more uniformly.

If you supply a non-power-of-2 capacity, HashMap rounds it up to the next power of 2 internally.

HIGH PRIORITY

What is the significance of the load factor?

Typically asked as: Scenario-Based

The load factor (default 0.75) determines when HashMap resizes. It represents a trade-off:

- Higher load factor (e.g., 1.0): Less memory waste, but more collisions and slower lookups
- Lower load factor (e.g., 0.5): Faster lookups with fewer collisions, but more memory usage due to earlier resizing

The default 0.75 is a well-tested compromise between time and space cost. In practice, you rarely need to change it unless you have specific memory or performance constraints.

HIGH PRIORITY

Explain the treeification threshold in Java 8 HashMap.

Typically asked as: Follow-up

When a bucket's linked list grows to 8 or more nodes AND the total HashMap capacity is at least 64, the linked list is converted to a red-black tree. This improves worst-case lookup from O(n) to O(log n).

The reverse (untreeify) happens when the tree shrinks below 6 nodes during resize. The gap between 8 (treeify) and 6 (untreeify) prevents rapid alternation between tree and list structures.

```
// Constants from HashMap source
static final int TREEIFY_THRESHOLD = 8;
static final int UNTREEIFY_THRESHOLD = 6;
static final int MIN_TREEIFY_CAPACITY = 64;
```

MEDIUM PRIORITY

Can we use a mutable object as a HashMap key?

Typically asked as: Tricky

Technically yes, but it is extremely dangerous. If you modify the key after insertion, its hashCode() changes, meaning the bucket index changes. The entry becomes unreachable - get() will look in the wrong bucket and return null, yet the entry still occupies memory.

Rule: Always use immutable objects (String, Integer, or custom immutables) as HashMap keys.

```
List<String> key = new ArrayList<>(Arrays.asList("hello"));
Map<List<String>, String> map = new HashMap<>();
map.put(key, "value");
System.out.println(map.get(key)); // "value"

key.add("world"); // MUTATING THE KEY
System.out.println(map.get(key)); // null - entry is lost!
```

MUST KNOW

HashMap vs ConcurrentHashMap

This comparison is asked in nearly every interview that touches Collections or Multithreading. Interviewers expect you to know the internal concurrency mechanism, not just "it's thread-safe."

Key Differences:

Aspect	HashMap	ConcurrentHashMap
Thread Safety	Not thread-safe	Thread-safe
Null keys/values	One null key, multiple null values	No nulls allowed
Locking	None	Segment-level (Java 7) / Node-level + CAS (Java 8+)
Iterator	Fail-fast	Weakly consistent
Performance (single thread)	Faster	Slight overhead
Performance (concurrent)	Breaks under concurrency	Designed for it

Java 8 ConcurrentHashMap Internals:

Java 8 replaced segment-based locking with a finer-grained approach:

- Uses CAS (Compare-And-Swap) for inserting into empty buckets
- Uses synchronized on the first node of a bucket for collision cases
- Result: Multiple threads can write to different buckets simultaneously without blocking

```
// ConcurrentHashMap allows safe concurrent updates
ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();

// atomic compute operations - no external synchronization needed
map.compute("count", (key, val) -> val == null ? 1 : val + 1);
map.putIfAbsent("key", 0);
map.merge("count", 1, Integer::sum);
```

MUST KNOW

Why does ConcurrentHashMap not allow null keys or values?

Typically asked as: Most Frequently Asked

Because null creates ambiguity in concurrent contexts. If map.get(key) returns null, you cannot tell whether:

- The key is absent from the map, OR
- The key is present but mapped to null

In a single-threaded HashMap, you can resolve this by calling containsKey(). But in a concurrent context, another thread might insert or remove the key between your get() and containsKey() calls, making the check unreliable.

HIGH PRIORITY

What is the difference between Collections.synchronizedMap() and ConcurrentHashMap?

Typically asked as: Scenario-Based

Collections.synchronizedMap() wraps a HashMap with a single lock on the entire map. Every operation (read or write) acquires the same lock, meaning only one thread can access the map at a time - even for reads.

ConcurrentHashMap uses fine-grained locking. Multiple reads can proceed concurrently with no locking at all (reads are lock-free in Java 8). Writes only lock individual buckets. This provides dramatically better throughput under contention.

When to use which:

- synchronizedMap: When you have a pre-existing HashMap that needs basic thread safety with minimal code change
- ConcurrentHashMap: When you design for concurrency from the start and need high throughput

MUST KNOW

equals() and hashCode()

The equals() and hashCode() contract is foundational to Collections and is asked in nearly every Java interview. A broken contract causes silent, hard-to-debug bugs in HashMaps and HashSets.

The Contract:

1. If a.equals(b) is true, then a.hashCode() == b.hashCode() must be true

2. If a.hashCode() != b.hashCode(), then a.equals(b) must be false

3. The reverse is NOT required: two objects with the same hashCode are NOT required to be equal (this is a collision)

```
public class Student {
    private int rollNumber;
    private String name;

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Student student = (Student) o;
        return rollNumber == student.rollNumber
            && Objects.equals(name, student.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(rollNumber, name);
    }
}
```

MUST KNOW

What happens if you override equals() but not hashCode()?

Typically asked as: Most Frequently Asked

The object will behave incorrectly in hash-based collections (HashMap, HashSet, Hashtable). Two logically equal objects will have different hash codes, so they will be placed in different buckets. This means:

- A HashSet can contain "duplicate" objects that are logically equal
- map.get(equalKey) returns null even though an equal key exists in the map

This is one of the most common bugs in Java applications and a favorite interview question.

```
// BUG: equals overridden but hashCode not
class BrokenKey {
    int id;
    public boolean equals(Object o) {
        return o instanceof BrokenKey && ((BrokenKey) o).id == this.id;
    }
    // hashCode() NOT overridden - uses Object.hashCode() (memory address)
}

BrokenKey k1 = new BrokenKey(); k1.id = 1;
BrokenKey k2 = new BrokenKey(); k2.id = 1;

Map<BrokenKey, String> map = new HashMap<>();
map.put(k1, "hello");
System.out.println(map.get(k2)); // null! k1.equals(k2) but different hashCodes
```

HIGH PRIORITY

Why should the fields used in equals() also be used in hashCode()?

Typically asked as: Follow-up

To maintain the contract. If equals() considers fields A and B, but hashCode() only uses field A, then two objects with the same A but different B would:

- Be considered NOT equal by equals() (correct)
- Have the SAME hashCode (not a bug, but causes excessive collisions)

Conversely, if hashCode() uses a field that equals() ignores, you get objects where equals() returns true but hashCode() differs - this violates the contract and breaks HashMap.

The safest practice: use exactly the same set of fields in both methods.

HIGH PRIORITY

List Implementations

MUST KNOW

What are the main List implementations and when do you use each?

Typically asked as: Most Frequently Asked

Implementation	Underlying Structure	Best For
ArrayList	Dynamic array	Random access, iteration, most use cases
LinkedList	Doubly-linked list	Frequent insertion/deletion at both ends
Vector	Synchronized dynamic array	Legacy code (avoid in new code)
CopyOnWriteArrayList	Copy-on-write array	Read-heavy concurrent scenarios

In practice, ArrayList is the right choice 95% of the time. LinkedList is rarely faster in real workloads due to CPU cache effects and pointer-chasing overhead.

```
// ArrayList - most common choice
List<String> names = new ArrayList<>();
names.add("Alice");       // O(1) amortized
names.get(0);             // O(1) random access
names.remove(0);          // O(n) - shifts elements

// LinkedList - only when you need Deque operations
Deque<String> queue = new LinkedList<>();
queue.offerFirst("first");  // O(1)
queue.pollLast();           // O(1)
```

HIGH PRIORITY

How does ArrayList grow internally?

Typically asked as: Follow-up

ArrayList starts with a default capacity of 10. When size exceeds capacity:

1. A new array is created with size oldCapacity + (oldCapacity >> 1) - approximately 1.5x the old capacity
2. All existing elements are copied to the new array using Arrays.copyOf()
3. The old array becomes eligible for garbage collection

This means add() is O(1) amortized but O(n) in the worst case (during resize). If you know the final size, use new ArrayList<>(expectedSize) to avoid resizing.

HIGH PRIORITY

Set Implementations

MUST KNOW

How does HashSet ensure uniqueness?

Typically asked as: Most Frequently Asked

HashSet is backed by a HashMap internally. When you add an element:

- The element becomes the KEY in the internal HashMap
- A dummy constant object (PRESENT) is the VALUE
- Since HashMap keys must be unique (determined by equals() and hashCode()), the Set automatically rejects duplicates

```
// Internal implementation (simplified)
public class HashSet<E> {
    private transient HashMap<E, Object> map;
    private static final Object PRESENT = new Object();

    public boolean add(E e) {
        return map.put(e, PRESENT) == null;
    }
}
```

This is why you must override both equals() and hashCode() for custom objects stored in a HashSet.

HIGH PRIORITY

What is the difference between HashSet, LinkedHashSet, and TreeSet?

Typically asked as: Scenario-Based

Feature	HashSet	LinkedHashSet	TreeSet
Ordering	No guaranteed order	Insertion order	Sorted (natural or Comparator)
Null elements	One null allowed	One null allowed	No null (throws NPE)
Performance	O(1) add/remove/contains	O(1) with slight overhead	O(log n)
Underlying	HashMap	LinkedHashMap	TreeMap (Red-Black Tree)

Use TreeSet when you need elements automatically sorted. Use LinkedHashSet when you need predictable iteration order matching insertion sequence.

HIGH PRIORITY

Map Implementations

HIGH PRIORITY

Compare HashMap, LinkedHashMap, and TreeMap.

Typically asked as: Most Frequently Asked

Feature	HashMap	LinkedHashMap	TreeMap
Ordering	No order	Insertion order (or access order)	Sorted by key
Null keys	One null key	One null key	No null keys
Performance	O(1)	O(1) with slight overhead	O(log n)
Use case	General purpose	LRU cache, ordered iteration	Range queries, sorted data

```
// LinkedHashMap as LRU Cache
Map<String, String> lruCache = new LinkedHashMap<>(16, 0.75f, true) {
    @Override
    protected boolean removeEldestEntry(Map.Entry<String, String> eldest) {
        return size() > 100; // keep max 100 entries
    }
};
```

MEDIUM PRIORITY

When would you use TreeMap over HashMap?

Typically asked as: Scenario-Based

Use TreeMap when you need:

- Keys in sorted order (natural ordering or custom Comparator)
- Range operations: subMap(), headMap(), tailMap()
- Finding nearest keys: floorKey(), ceilingKey(), higherKey(), lowerKey()

Trade-off: O(log n) for put/get versus O(1) with HashMap. TreeMap uses a Red-Black tree internally.

MUST KNOW

Comparable vs Comparator

MUST KNOW

What is the difference between Comparable and Comparator?

Typically asked as: Most Frequently Asked

Aspect	Comparable	Comparator
Package	`java.lang`	`java.util`
Method	`compareTo(T o)`	`compare(T o1, T o2)`
Modifies class	Yes (implements interface)	No (external)
Number of orderings	One (natural order)	Multiple (create different Comparators)
Usage	`Collections.sort(list)`	`Collections.sort(list, comparator)`

```
// Comparable - natural ordering built into the class
public class Employee implements Comparable<Employee> {
    private String name;
    private int salary;

    @Override
    public int compareTo(Employee other) {
        return Integer.compare(this.salary, other.salary);
    }
}

// Comparator - external, multiple orderings possible
Comparator<Employee> byName = Comparator.comparing(Employee::getName);
Comparator<Employee> bySalaryDesc = Comparator.comparingInt(Employee::getSalary).reversed();

list.sort(byName.thenComparing(bySalaryDesc));
```

HIGH PRIORITY

When should you use Comparable vs Comparator?

Typically asked as: Scenario-Based

Use Comparable when there is one obvious natural ordering for a class (e.g., String alphabetical, Integer numeric). Use Comparator when:

- You need multiple different orderings
- You cannot modify the class (third-party library)
- The ordering is context-specific (sort by name in one place, by age elsewhere)

In modern Java (8+), prefer Comparator with method references: Comparator.comparing(Employee::getName).

HIGH PRIORITY

Collection vs Collections

HIGH PRIORITY

What is the difference between Collection and Collections?

Typically asked as: Most Frequently Asked

- Collection (interface): The root interface of the Collections framework. Extended by List, Set, Queue.

Defines basic operations like add(), remove(), contains(), iterator().

- Collections (utility class): A final class with static utility methods for operating on collections - sort(), reverse(), unmodifiableList(), synchronizedMap(), singleton(), etc.

```
// Collection - the interface
Collection<String> items = new ArrayList<>();
items.add("one");

// Collections - the utility class
List<String> list = new ArrayList<>(Arrays.asList("c", "a", "b"));
Collections.sort(list);                    // [a, b, c]
List<String> immutable = Collections.unmodifiableList(list);
List<String> synced = Collections.synchronizedList(list);
```

HIGH PRIORITY

Iterator vs ListIterator

HIGH PRIORITY

What is the difference between Iterator and ListIterator?

Typically asked as: Most Frequently Asked

Feature	Iterator	ListIterator
Direction	Forward only	Both forward and backward
Applicable to	Any Collection	Only List
Operations	`hasNext()`, `next()`, `remove()`	+ `hasPrevious()`, `previous()`, `add()`, `set()`, `nextIndex()`, `previousIndex()`
Obtain from	`collection.iterator()`	`list.listIterator()`

```
List<String> list = new ArrayList<>(Arrays.asList("A", "B", "C"));

// ListIterator - bidirectional with modification
ListIterator<String> lit = list.listIterator();
lit.next();       // "A"
lit.set("X");     // replaces "A" with "X"
lit.add("Y");     // inserts "Y" after current position
lit.previous();   // "Y"
```

MUST KNOW

Fail-Fast vs Fail-Safe Iterators

MUST KNOW

What are fail-fast and fail-safe iterators?

Typically asked as: Most Frequently Asked

Fail-Fast (ArrayList, HashMap, HashSet): Immediately throws ConcurrentModificationException if the collection is structurally modified while iterating (except through the iterator's own remove() method). Uses an internal modCount counter - if modCount changes unexpectedly, the iterator fails.

Fail-Safe (ConcurrentHashMap, CopyOnWriteArrayList): Works on a copy or snapshot of the collection. Never throws ConcurrentModificationException. Trade-off: may not reflect the most recent modifications.

```
// Fail-Fast - throws ConcurrentModificationException
List<String> list = new ArrayList<>(Arrays.asList("A", "B", "C"));
for (String s : list) {
    if (s.equals("B")) {
        list.remove(s); // THROWS ConcurrentModificationException
    }
}

// Correct way - use Iterator.remove()
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    if (it.next().equals("B")) {
        it.remove(); // safe
    }
}

// Fail-Safe - no exception
ConcurrentHashMap<String, Integer> cmap = new ConcurrentHashMap<>();
cmap.put("A", 1);
cmap.put("B", 2);
for (String key : cmap.keySet()) {
    cmap.put("C", 3); // no exception, but "C" may or may not appear in iteration
}
```

HIGH PRIORITY

How does fail-fast detection work internally?

Typically asked as: Follow-up

ArrayList (and similar collections) maintain an integer field called modCount. Every structural modification (add, remove, clear) increments this counter. When an iterator is created, it saves the current modCount as expectedModCount. On each next() call, the iterator checks if modCount == expectedModCount. If not, it throws ConcurrentModificationException.

Note: This is a "best-effort" mechanism, not a guarantee. It cannot detect all concurrent modifications in a multithreaded environment without external synchronization.

MUST KNOW

ArrayList vs LinkedList

MUST KNOW

When would you choose LinkedList over ArrayList?

Typically asked as: Most Frequently Asked

Almost never in practice. Despite the theoretical O(1) advantage for insertion/deletion at arbitrary positions, LinkedList is slower than ArrayList for most real workloads because:

1. CPU cache locality: ArrayList stores elements in contiguous memory, enabling hardware prefetching. LinkedList nodes are scattered across the heap.

2. Memory overhead: Each LinkedList node has two extra pointers (prev, next), approximately 24 bytes overhead per element.

3. Random access: ArrayList is O(1); LinkedList is O(n) - must traverse from head or tail.

Only use LinkedList when:

- You need a Deque (double-ended queue) with frequent addFirst()/removeFirst()/addLast()/removeLast()
- You never need random access by index
- The list is very large and you insert/remove at known positions using an iterator

Operation	ArrayList	LinkedList
`get(index)`	O(1)	O(n)
`add(end)`	O(1) amortized	O(1)
`add(index)`	O(n) - shift	O(1) if at iterator position
`remove(index)`	O(n) - shift	O(1) if at iterator position
Memory per element	~4 bytes (reference)	~24 bytes (node + pointers)

MEDIUM PRIORITY

TreeMap

MEDIUM PRIORITY

How does TreeMap maintain sorting?

Typically asked as: Most Frequently Asked

TreeMap uses a Red-Black Tree (a self-balancing binary search tree). Every insertion, deletion, and lookup is O(log n). Keys must either implement Comparable or a Comparator must be provided at construction time.

```
// Natural ordering
TreeMap<String, Integer> map = new TreeMap<>();
map.put("banana", 2);
map.put("apple", 1);
map.put("cherry", 3);
// Iteration order: apple, banana, cherry (sorted)

// Range queries
map.subMap("apple", "cherry"); // {apple=1, banana=2}
map.headMap("banana");          // {apple=1}
map.tailMap("banana");          // {banana=2, cherry=3}
```

HIGH PRIORITY

HashSet

HIGH PRIORITY

How does HashSet handle duplicate detection?

Typically asked as: Most Frequently Asked

HashSet delegates to HashMap internally. When add(element) is called:

1. element.hashCode() determines the bucket

2. If the bucket is empty -> element is added (not duplicate)

3. If the bucket is occupied -> element.equals(existingElement) is checked for each entry in the chain

4. If equals returns true for any existing entry -> element is NOT added (duplicate)

5. If equals returns false for all -> element is added to the chain

This is why custom objects in a HashSet must override both hashCode() and equals() - without both, the duplicate detection mechanism is broken.

HIGH PRIORITY

Concurrent Collections

HIGH PRIORITY

What are the main concurrent collection classes and when do you use them?

Typically asked as: Most Frequently Asked

Class	Replaces	Mechanism
ConcurrentHashMap	HashMap / Hashtable	CAS + node-level sync
CopyOnWriteArrayList	ArrayList (synced)	Copy-on-write for mutations
CopyOnWriteArraySet	HashSet (synced)	Copy-on-write
ConcurrentLinkedQueue	LinkedList (as queue)	Lock-free CAS
BlockingQueue (interface)	-	Producer-consumer pattern
ConcurrentSkipListMap	TreeMap (synced)	Lock-free skip list

```
// CopyOnWriteArrayList - ideal for read-heavy, write-rare scenarios
CopyOnWriteArrayList<String> list = new CopyOnWriteArrayList<>();
list.add("A");
// Every write creates a new internal array - expensive for writes
// But reads never block and never throw ConcurrentModificationException

// BlockingQueue - producer-consumer
BlockingQueue<Task> queue = new ArrayBlockingQueue<>(100);
queue.put(task);        // blocks if full
Task t = queue.take();  // blocks if empty
```

MEDIUM PRIORITY

Why is CopyOnWriteArrayList expensive for writes?

Typically asked as: Follow-up

Every mutation (add, set, remove) creates a fresh copy of the entire underlying array. This means:

- Write operations are O(n) in both time and memory
- Reads are guaranteed to see a consistent snapshot - no locking needed
- Ideal when reads vastly outnumber writes (e.g., event listener lists)

Part 2 - Java 8

Java 8 features (Streams, Lambdas, Functional Interfaces) are the second most asked category. Every modern Java interview expects you to demonstrate fluency with the Stream API. Many interviewers present coding problems and expect a Stream-based solution.

MUST KNOW

Lambda Expressions

A lambda expression is a concise way to represent an anonymous function - a function that has no name, belongs to no class, and can be passed as an argument or stored in a variable.

Syntax: (parameters) -> expression or (parameters) -> { statements; }

```
// Before Java 8 - anonymous inner class
Runnable r1 = new Runnable() {
    @Override
    public void run() {
        System.out.println("Hello");
    }
};

// Java 8 Lambda - same thing, much cleaner
Runnable r2 = () -> System.out.println("Hello");

// With parameters
Comparator<String> comp = (a, b) -> a.length() - b.length();

// Multi-statement body
Consumer<String> printer = s -> {
    String upper = s.toUpperCase();
    System.out.println(upper);
};
```

MUST KNOW

What is the scope of variables in a lambda?

Typically asked as: Most Frequently Asked

Lambdas can access:

- Parameters of the lambda itself
- Local variables from the enclosing scope - but only if they are effectively final (not reassigned after initialization)
- Instance and static variables of the enclosing class (no restriction)

The "effectively final" restriction exists because lambdas may execute on a different thread or at a later time. If the variable could change, the lambda would see an inconsistent value.

```
int multiplier = 3; // effectively final - never reassigned
Function<Integer, Integer> multiply = x -> x * multiplier;

// This would NOT compile:
// multiplier = 5; // makes it non-effectively-final
```

HIGH PRIORITY

Can a lambda throw a checked exception?

Typically asked as: Tricky

Only if the functional interface's abstract method declares it in its throws clause. Standard functional interfaces (Function, Consumer, Predicate) do NOT declare checked exceptions, so you cannot throw one directly.

Workaround: wrap the checked exception in an unchecked exception, or create a custom functional interface.

```
// Custom functional interface that allows checked exceptions
@FunctionalInterface
interface ThrowingFunction<T, R> {
    R apply(T t) throws Exception;
}
```

MUST KNOW

Functional Interfaces

A functional interface has exactly one abstract method. It can have any number of default or static methods. The @FunctionalInterface annotation is optional but recommended - it causes a compile error if the contract is violated.

Key Built-in Functional Interfaces:

Interface	Method	Signature	Use Case
`Predicate<T>`	`test`	`T -> boolean`	Filtering
`Function<T,R>`	`apply`	`T -> R`	Transformation
`Consumer<T>`	`accept`	`T -> void`	Side effects
`Supplier<T>`	`get`	`() -> T`	Lazy generation
`UnaryOperator<T>`	`apply`	`T -> T`	Same-type transformation
`BinaryOperator<T>`	`apply`	`(T, T) -> T`	Reduction
`BiFunction<T,U,R>`	`apply`	`(T, U) -> R`	Two-arg transformation
`BiPredicate<T,U>`	`test`	`(T, U) -> boolean`	Two-arg filtering

```
// Predicate - filtering
Predicate<String> nonEmpty = s -> s != null && !s.isEmpty();
Predicate<String> startsWithA = s -> s.startsWith("A");
Predicate<String> combined = nonEmpty.and(startsWithA);

// Function - transformation
Function<String, Integer> length = String::length;
Function<String, String> shout = s -> s.toUpperCase() + "!";
Function<String, String> greet = s -> "Hello, " + s;
Function<String, String> greetAndShout = greet.andThen(shout);
```

MUST KNOW

What is the difference between Predicate and Function?

Typically asked as: Most Frequently Asked

Predicate<T> takes an input of type T and returns a boolean. It represents a condition or filter. Function<T, R> takes an input of type T and returns a result of type R (any type). Predicate is essentially Function<T, Boolean> but specialized for boolean logic with additional methods like and(), or(), negate().

HIGH PRIORITY

Can you create your own functional interface?

Typically asked as: Follow-up

Yes. Any interface with exactly one abstract method qualifies. Annotate with @FunctionalInterface for safety.

```
@FunctionalInterface
public interface TriFunction<A, B, C, R> {
    R apply(A a, B b, C c);
}

// Usage
TriFunction<Integer, Integer, Integer, Integer> sum = (a, b, c) -> a + b + c;
```

MUST KNOW

Streams

The Stream API processes collections of data declaratively - describing WHAT to compute rather than HOW. Streams are lazy (intermediate operations are not executed until a terminal operation is called), single-use, and optionally parallelizable.

Stream Pipeline Structure:

1. Source: Collection, array, generator, I/O channel
2. Intermediate operations (lazy): filter, map, flatMap, sorted, distinct, peek, limit, skip
3. Terminal operation (triggers execution): collect, forEach, reduce, count, findFirst, anyMatch, toArray

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie", "David", "Eve");

// Complete pipeline: source -> intermediate -> terminal
List<String> result = names.stream()          // source
    .filter(n -> n.length() > 3)              // intermediate
    .map(String::toUpperCase)                  // intermediate
    .sorted()                                  // intermediate
    .collect(Collectors.toList());             // terminal
// Result: [ALICE, CHARLIE, DAVID]
```

MUST KNOW

What is the difference between intermediate and terminal operations?

Typically asked as: Most Frequently Asked

Intermediate operations are lazy - they return a new Stream and do nothing until a terminal operation is invoked. They can be chained. Examples: filter(), map(), sorted().

Terminal operations trigger the actual processing of the pipeline and produce a result or side effect. After a terminal operation, the stream is consumed and cannot be reused. Examples: collect(), forEach(), count(), reduce().

MUST KNOW

Can a stream be reused?

Typically asked as: Tricky

No. Once a terminal operation is invoked, the stream is consumed. Attempting to use it again throws IllegalStateException. You must create a new stream from the source.

```
Stream<String> stream = names.stream();
stream.forEach(System.out::println); // OK
stream.forEach(System.out::println); // IllegalStateException!
```

HIGH PRIORITY

What is lazy evaluation in streams?

Typically asked as: Follow-up

Intermediate operations are not executed until a terminal operation is called. This enables optimizations:

- Short-circuiting: findFirst() stops processing after finding one match
- Loop fusion: Multiple operations are applied in a single pass over data
- Unnecessary work elimination: If a terminal operation like count() doesn't need sorted data, the sort may be skipped in some cases

```
// Only "Alice" is processed through map() - findFirst short-circuits
Optional<String> first = names.stream()
    .filter(n -> n.startsWith("A"))
    .map(n -> {
        System.out.println("Mapping: " + n); // printed only once
        return n.toUpperCase();
    })
    .findFirst();
```

MUST KNOW

Stream Interview Questions

MUST KNOW

Find the second-highest salary from a list of employees.

Typically asked as: Coding

```
Optional<Integer> secondHighest = employees.stream()
    .map(Employee::getSalary)
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst();
```

HIGH PRIORITY

Group employees by department and find the highest-paid in each.

Typically asked as: Coding

```
Map<String, Optional<Employee>> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.maxBy(Comparator.comparingInt(Employee::getSalary))
    ));
```

HIGH PRIORITY

Find duplicate elements in a list using streams.

Typically asked as: Coding

```
Set<Integer> duplicates = numbers.stream()
    .filter(n -> Collections.frequency(numbers, n) > 1)
    .collect(Collectors.toSet());

// More efficient approach using a seen-set
Set<Integer> seen = new HashSet<>();
Set<Integer> duplicates2 = numbers.stream()
    .filter(n -> !seen.add(n))
    .collect(Collectors.toSet());
```

HIGH PRIORITY

Convert a list of strings to a comma-separated string.

Typically asked as: Coding

```
String csv = names.stream()
    .collect(Collectors.joining(", "));
// "Alice, Bob, Charlie, David, Eve"
```

MUST KNOW

Optional

Optional is a container that may or may not hold a non-null value. It was introduced to reduce NullPointerException and make APIs more expressive about nullable returns.

```
// Creating Optional
Optional<String> present = Optional.of("hello");       // non-null
Optional<String> empty = Optional.empty();              // empty
Optional<String> nullable = Optional.ofNullable(null);  // empty if null

// Consuming Optional - NEVER call get() without checking
String result = present
    .filter(s -> s.length() > 3)
    .map(String::toUpperCase)
    .orElse("default");

// With fallback computation
String computed = empty.orElseGet(() -> expensiveComputation());

// Throw if absent
String required = empty.orElseThrow(
    () -> new IllegalStateException("Value required"));
```

HIGH PRIORITY

When should you NOT use Optional?

Typically asked as: Scenario-Based

- Never as a method parameter - use overloading or nullable parameters instead
- Never as a field type - Optional is not Serializable and adds overhead
- Never in collections - List<Optional<T>> is an anti-pattern; filter out nulls instead
- Use it as a return type from methods that might legitimately have no result

MUST KNOW

What is the difference between orElse() and orElseGet()?

Typically asked as: Most Frequently Asked

orElse(value) always evaluates the fallback value, even if Optional is present. orElseGet(supplier) only evaluates the supplier if Optional is empty.

```
// orElse - expensiveMethod() is ALWAYS called
String r1 = optional.orElse(expensiveMethod());

// orElseGet - expensiveMethod() called ONLY if empty
String r2 = optional.orElseGet(() -> expensiveMethod());
```

This matters when the fallback involves a costly operation (database call, network request, object creation).

MUST KNOW

Method References

Method references are shorthand for lambdas that simply call an existing method. They improve readability without changing behavior.

Four types:

Type	Syntax	Lambda Equivalent
Static method	`ClassName::staticMethod`	`x -> ClassName.staticMethod(x)`
Instance method (bound)	`instance::method`	`x -> instance.method(x)`
Instance method (unbound)	`ClassName::method`	`(obj, x) -> obj.method(x)`
Constructor	`ClassName::new`	`x -> new ClassName(x)`

```
// Static method reference
Function<String, Integer> parse = Integer::parseInt;

// Bound instance method
String str = "hello";
Supplier<Integer> len = str::length;

// Unbound instance method
Function<String, Integer> length = String::length;

// Constructor reference
Supplier<ArrayList<String>> listFactory = ArrayList::new;
Function<Integer, int[]> arrayFactory = int[]::new;
```

MUST KNOW

Collectors

The Collectors utility class provides reduction operations for use with Stream.collect(). These are the most commonly tested collector methods.

MUST KNOW

What are the most important Collectors methods?

Typically asked as: Most Frequently Asked

```
List<Employee> employees = getEmployees();

// toList(), toSet(), toMap()
List<String> names = employees.stream()
    .map(Employee::getName)
    .collect(Collectors.toList());

Map<Integer, String> idToName = employees.stream()
    .collect(Collectors.toMap(Employee::getId, Employee::getName));

// joining
String allNames = employees.stream()
    .map(Employee::getName)
    .collect(Collectors.joining(", ", "[", "]"));

// counting, summarizing
long count = employees.stream().collect(Collectors.counting());
IntSummaryStatistics stats = employees.stream()
    .collect(Collectors.summarizingInt(Employee::getSalary));
```

MUST KNOW

map() vs flatMap()

MUST KNOW

What is the difference between map() and flatMap()?

Typically asked as: Most Frequently Asked

map() applies a function to each element and wraps the result - one input produces one output. flatMap() applies a function that returns a stream for each element, then flattens all resulting streams into a single stream - one input can produce zero, one, or many outputs.

```
// map - 1:1 transformation
List<String> words = Arrays.asList("hello", "world");
List<Integer> lengths = words.stream()
    .map(String::length)
    .collect(Collectors.toList()); // [5, 5]

// flatMap - 1:many, then flatten
List<List<Integer>> nested = Arrays.asList(
    Arrays.asList(1, 2), Arrays.asList(3, 4));
List<Integer> flat = nested.stream()
    .flatMap(Collection::stream)
    .collect(Collectors.toList()); // [1, 2, 3, 4]

// Practical example: get all characters from all words
List<String> chars = words.stream()
    .flatMap(w -> Arrays.stream(w.split("")))
    .distinct()
    .collect(Collectors.toList());
```

HIGH PRIORITY

reduce()

HIGH PRIORITY

How does reduce() work?

Typically asked as: Most Frequently Asked

reduce() combines all elements of a stream into a single result by repeatedly applying a binary operator.

Three forms:

1. reduce(BinaryOperator) - returns Optional<T>

2. reduce(identity, BinaryOperator) - returns T

3. reduce(identity, BiFunction, BinaryOperator) - for parallel streams with different accumulator/combiner types

```
// Sum all numbers
int sum = numbers.stream().reduce(0, Integer::sum);

// Find max
Optional<Integer> max = numbers.stream().reduce(Integer::max);

// Concatenate strings
String joined = words.stream().reduce("", (a, b) -> a + " " + b).trim();
```

MUST KNOW

groupingBy() and partitioningBy()

MUST KNOW

What is the difference between groupingBy and partitioningBy?

Typically asked as: Most Frequently Asked

groupingBy() groups elements into a Map<K, List<T>> by a classifier function - can produce any number of groups. partitioningBy() splits elements into exactly two groups (Map<Boolean, List<T>>) based on a predicate - true group and false group.

```
// groupingBy - multiple groups
Map<String, List<Employee>> byDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment));

// groupingBy with downstream collector
Map<String, Long> countByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment, Collectors.counting()));

// partitioningBy - exactly two groups
Map<Boolean, List<Employee>> partitioned = employees.stream()
    .collect(Collectors.partitioningBy(e -> e.getSalary() > 50000));
List<Employee> highEarners = partitioned.get(true);
List<Employee> others = partitioned.get(false);
```

MEDIUM PRIORITY

Parallel Streams

MEDIUM PRIORITY

When should you use parallel streams?

Typically asked as: Scenario-Based

Parallel streams split work across multiple threads using the common ForkJoinPool. Use them when:

- The data source is large (thousands of elements minimum)
- Operations are stateless and CPU-intensive
- The source supports efficient splitting (ArrayList, arrays - NOT LinkedList)
- No shared mutable state is accessed during processing

Avoid parallel streams when:

- Data set is small (thread coordination overhead dominates)
- Operations involve I/O (threads block, exhausting the pool)
- Order matters and you use forEachOrdered() (negates parallelism)
- Operations are stateful or have side effects

```
// Good candidate: CPU-intensive, large dataset, splitable source
long sum = LongStream.rangeClosed(1, 10_000_000)
    .parallel()
    .reduce(0L, Long::sum);

// BAD: shared mutable state
List<Integer> results = new ArrayList<>(); // NOT thread-safe
numbers.parallelStream()
    .map(n -> n * 2)
    .forEach(results::add); // RACE CONDITION - use collect() instead
```

Part 3 - OOP Concepts

Object-Oriented Programming concepts form the foundation of Java and are asked in every interview. Interviewers focus on practical understanding - how you apply these concepts in real projects, not textbook definitions.

MUST KNOW

Encapsulation

Encapsulation is the practice of hiding internal state and exposing controlled access through methods. It protects data integrity and allows implementation changes without affecting external code.

```
public class BankAccount {
    private double balance; // hidden state

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("Amount must be positive");
        this.balance += amount;
    }

    public void withdraw(double amount) {
        if (amount > balance) throw new InsufficientFundsException();
        this.balance -= amount;
    }
}
```

MUST KNOW

What is encapsulation and why is it important?

Typically asked as: Most Frequently Asked

Encapsulation bundles data and the methods that operate on that data into a single unit (class), while restricting direct access to the data. Benefits:

- Data integrity: Validation logic in setters prevents invalid state
- Flexibility: Internal implementation can change without breaking clients
- Reduced coupling: External code depends on the public API, not internal fields
- Better testability: Controlled access points make behavior predictable

HIGH PRIORITY

What is the difference between encapsulation and abstraction?

Typically asked as: Follow-up

Encapsulation is about hiding implementation details (HOW something works). Abstraction is about hiding complexity and showing only relevant features (WHAT something does). Encapsulation is the mechanism;

abstraction is the design goal. A well-encapsulated class provides abstraction to its clients.

MUST KNOW

Inheritance

Inheritance establishes an IS-A relationship between classes. A subclass inherits all non-private fields and methods from its superclass and can extend or override them.

```
public class Shape {
    protected String color;

    public double area() {
        return 0;
    }
}

public class Circle extends Shape {
    private double radius;

    @Override
    public double area() {
        return Math.PI * radius * radius;
    }
}
```

MUST KNOW

Why does Java not support multiple inheritance of classes?

Typically asked as: Most Frequently Asked

Multiple inheritance creates the Diamond Problem: if class D extends both B and C, and both B and C override a method from A, which version does D inherit? This ambiguity leads to unpredictable behavior.

Java avoids this with single inheritance for classes but allows multiple inheritance of type through interfaces. Since Java 8, interfaces can have default methods, which re-introduces a mild form of the diamond problem - resolved by requiring the implementing class to explicitly override the conflicting method.

HIGH PRIORITY

What is method hiding vs method overriding?

Typically asked as: Tricky

Overriding applies to instance methods - the JVM dispatches to the actual object's type at runtime (dynamic polymorphism). Hiding applies to static methods - the compiler resolves based on the reference type at compile time.

```
class Parent {
    static void staticMethod() { System.out.println("Parent"); }
    void instanceMethod() { System.out.println("Parent"); }
}
class Child extends Parent {
    static void staticMethod() { System.out.println("Child"); } // hiding
    void instanceMethod() { System.out.println("Child"); }      // overriding
}

Parent p = new Child();
p.staticMethod();   // "Parent" - resolved by reference type (hiding)
p.instanceMethod(); // "Child"  - resolved by object type (overriding)
```

MUST KNOW

Polymorphism

Polymorphism means "one interface, multiple implementations." In Java it manifests as:

- Compile-time (static) polymorphism: Method overloading
- Runtime (dynamic) polymorphism: Method overriding

MUST KNOW

What is runtime polymorphism and how does it work?

Typically asked as: Most Frequently Asked

Runtime polymorphism occurs when a method call on a superclass reference is resolved to the subclass implementation at runtime. The JVM uses a virtual method table (vtable) to determine which method to invoke based on the actual object type, not the reference type.

```
Animal animal = new Dog(); // reference: Animal, object: Dog
animal.speak(); // calls Dog.speak(), not Animal.speak()
```

This enables writing code against interfaces/abstract types while the actual behavior varies based on the concrete implementation provided at runtime.

HIGH PRIORITY

Can you override a private or static method?

Typically asked as: Tricky

- Private methods cannot be overridden - they are not visible to subclasses. A subclass can define a method with the same name, but it is a completely separate method.
- Static methods cannot be overridden - they are hidden, not overridden. See method hiding above.
- Final methods cannot be overridden - this is enforced by the compiler.

MUST KNOW

Abstraction

Abstraction hides complex implementation details and exposes only the relevant behavior through abstract classes or interfaces.

MUST KNOW

What is the difference between abstract class and interface?

Typically asked as: Most Frequently Asked

Feature	Abstract Class	Interface
Methods	Abstract + concrete	Abstract + default + static (Java 8+)
Fields	Any fields (instance, static)	Only `public static final` constants
Constructor	Yes	No
Inheritance	Single (`extends`)	Multiple (`implements`)
Access modifiers	Any	Methods are `public` by default
Use case	Shared base with common state	Contract/capability definition

Design guideline: Use an abstract class when subclasses share common state or behavior. Use an interface when you want to define a capability that multiple unrelated classes can implement.

```
// Abstract class - shared state + partial implementation
abstract class Vehicle {
    protected int speed;
    abstract void accelerate();
    void brake() { speed = Math.max(0, speed - 10); }
}

// Interface - capability contract
interface Flyable {
    void fly();
    default void land() { System.out.println("Landing..."); }
}
```

MUST KNOW

Interface vs Abstract Class

MUST KNOW

When would you choose interface over abstract class in a real project?

Typically asked as: Scenario-Based

Choose interface when:

- Multiple unrelated classes need the capability (e.g., Serializable, Comparable)

- You want to support multiple inheritance of type
- You are defining a pure contract without shared state

Choose abstract class when:

- Subclasses share significant common code or state
- You need constructors for initialization
- You want to enforce a template method pattern
- You need non-public methods shared among subclasses

Real-world example: In a payment system, PaymentProcessor is an interface (Stripe, PayPal, and BankTransfer are unrelated). But AbstractHttpClient is an abstract class (all HTTP clients share connection pooling, retry logic, and timeout handling).

HIGH PRIORITY

Association, Aggregation, and Composition

These describe relationships between objects - frequently tested with "explain with example" questions.

HIGH PRIORITY

What is the difference between aggregation and composition?

Typically asked as: Most Frequently Asked

Both represent HAS-A relationships, but differ in lifecycle dependency:

- Aggregation (weak HAS-A): The contained object can exist independently. If the container is destroyed, the contained object survives. Example: Department HAS Employees - employees continue to exist if the department is dissolved.

- Composition (strong HAS-A): The contained object cannot exist without the container. If the container is destroyed, the contained objects are destroyed too. Example: House HAS Rooms - rooms do not exist independently of the house.

```
// Composition - Engine is created and destroyed with Car
class Car {
    private final Engine engine; // lifecycle tied to Car
    Car() { this.engine = new Engine(); }
}

// Aggregation - Employee exists independently of Department
class Department {
    private List<Employee> employees; // employees can exist without department
    void addEmployee(Employee e) { employees.add(e); }
}
```

MEDIUM PRIORITY

Real Project Examples

MEDIUM PRIORITY

How do you apply OOP principles in a real Spring Boot project?

Typically asked as: Scenario-Based

In a typical Spring Boot e-commerce application:

- Encapsulation: Entity classes hide fields behind getters/setters with validation
- Inheritance: BaseEntity with common fields (id, createdAt, updatedAt) extended by all entities
- Polymorphism: PaymentService interface with CreditCardPayment, UPIPayment implementations - Spring injects the correct one based on configuration
- Abstraction: Repository interfaces hide database access details; services depend on abstractions, not concrete implementations
- Composition: Order composes OrderItem objects - deleting an order cascades to items
- Interface segregation: Separate Readable, Writable, Deletable interfaces rather than one fat CrudRepository

Part 4 - String

String is deceptively simple but hides important interview topics around immutability, memory management, and the String pool. Expect 2-3 String questions in most interviews.

MUST KNOW

String Pool and Immutability

MUST KNOW

Why is String immutable in Java?

Typically asked as: Most Frequently Asked

String immutability exists for three critical reasons:

1. String Pool optimization: The JVM maintains a pool of String literals in the heap (String Constant Pool). Multiple references can safely point to the same String object because it cannot be modified. Without immutability, changing one reference would corrupt all others.

2. Security: Strings are used for class loading, network connections, file paths, and database URLs. If Strings were mutable, an attacker could modify a validated path or hostname after security checks pass.

3. Thread safety: Immutable objects are inherently thread-safe - no synchronization needed when sharing String references across threads.

4. Hashcode caching: String caches its hashCode() value. Since the content never changes, the hash only needs to be computed once, making String an ideal HashMap key.

```
String s1 = "hello";       // goes to String Pool
String s2 = "hello";       // reuses same pool object
String s3 = new String("hello"); // creates new object on heap

System.out.println(s1 == s2);       // true  - same pool reference
System.out.println(s1 == s3);       // false - different objects
System.out.println(s1.equals(s3));  // true  - same content

String s4 = s3.intern(); // moves/finds in pool
System.out.println(s1 == s4);       // true  - both point to pool
```

MUST KNOW

How many objects are created by `new String("hello")`?

Typically asked as: Tricky

Up to two objects:

1. One String object in the String Pool (if "hello" literal was not already there)

2. One String object on the regular heap (the new keyword always creates a new object)

If "hello" already exists in the pool (from a previous literal usage), only one new object is created on the heap.

MUST KNOW

What is the difference between `==` and `equals()` for Strings?

Typically asked as: Most Frequently Asked

- == compares references - do both variables point to the same memory location?
- equals() compares content - do both Strings contain the same character sequence?

Always use equals() (or equalsIgnoreCase()) for String comparison. Using == works for literals from the pool but fails for new String() objects and Strings produced by methods like substring() or concat().

HIGH PRIORITY

StringBuilder vs StringBuffer

MUST KNOW

What is the difference between StringBuilder and StringBuffer?

Typically asked as: Most Frequently Asked

Feature	StringBuilder	StringBuffer
Thread safety	Not synchronized	Synchronized (thread-safe)
Performance	Faster	Slower (lock overhead)
Introduced	Java 1.5	Java 1.0
Use case	Single-threaded string building	Multi-threaded (rare in practice)

Always prefer StringBuilder unless you specifically need thread-safe string building (which is extremely rare - usually you build strings within a single method).

```
// StringBuilder - preferred for concatenation in loops
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 1000; i++) {
    sb.append("item").append(i).append(", ");
}
String result = sb.toString();

// Why not String concatenation in loops?
String bad = "";
for (int i = 0; i < 1000; i++) {
    bad += "item" + i; // creates ~1000 intermediate String objects
}
```

HIGH PRIORITY

Why is String concatenation with `+` in loops inefficient?

Typically asked as: Follow-up

Each + operation creates a new StringBuilder, appends, and calls toString() - producing a new String object on the heap. In a loop of N iterations, this creates approximately N intermediate String objects that immediately become garbage. StringBuilder avoids this by maintaining a single mutable buffer.

Note: Outside loops, the compiler optimizes "a" + "b" + "c" into a single StringBuilder chain, so single-line concatenation is fine.

HIGH PRIORITY

String Interview Questions

HIGH PRIORITY

How do you reverse a String in Java?

Typically asked as: Coding

```
// Using StringBuilder (recommended)
String reversed = new StringBuilder("hello").reverse().toString();

// Using char array (shows understanding)
char[] chars = original.toCharArray();
int left = 0, right = chars.length - 1;
while (left < right) {
    char temp = chars[left];
    chars[left++] = chars[right];
    chars[right--] = temp;
}
String reversed2 = new String(chars);
```

HIGH PRIORITY

How do you check if a String is a palindrome?

Typically asked as: Coding

```
public boolean isPalindrome(String s) {
    String clean = s.replaceAll("[^a-zA-Z0-9]", "").toLowerCase();
    return clean.equals(new StringBuilder(clean).reverse().toString());
}
```

MEDIUM PRIORITY

What does the intern() method do?

Typically asked as: Follow-up

intern() returns a canonical representation from the String Pool. If the pool already contains a String equal to this one (by equals()), that pooled reference is returned. Otherwise, the String is added to the pool and returned. This allows == comparison to work reliably for interned Strings.

Part 5 - Exception Handling

Exception handling questions test your understanding of Java's error model, the class hierarchy, and best practices. Expect 1-2 focused questions in most interviews.

MUST KNOW

Checked vs Unchecked Exceptions

MUST KNOW

What is the difference between checked and unchecked exceptions?

Typically asked as: Most Frequently Asked

Checked exceptions (compile-time): Must be either caught in a try-catch block or declared in the method signature with throws. The compiler enforces handling. They represent recoverable conditions. Examples: IOException, SQLException, FileNotFoundException.

Unchecked exceptions (runtime): Subclasses of RuntimeException. The compiler does not force you to handle them. They typically represent programming errors. Examples: NullPointerException, ArrayIndexOutOfBoundsException, IllegalArgumentException.

The class hierarchy:

- Throwable -> Error (unrecoverable, don't catch) + Exception
- Exception -> checked exceptions + RuntimeException (unchecked)

```
// Checked - compiler forces handling
public void readFile(String path) throws IOException {
    BufferedReader reader = new BufferedReader(new FileReader(path));
    // IOException is checked - must declare or catch
}

// Unchecked - compiler does not enforce
public int divide(int a, int b) {
    return a / b; // may throw ArithmeticException (unchecked)
}
```

HIGH PRIORITY

When should you use checked vs unchecked exceptions?

Typically asked as: Scenario-Based

Use checked exceptions when the caller can reasonably recover:

- File not found -> prompt user for different path
- Network timeout -> retry the request
- Invalid input from external source -> request corrected input

Use unchecked exceptions when it indicates a programming bug:

- Null pointer -> developer should have checked
- Array index out of bounds -> logic error
- Illegal argument -> caller violated API contract

Modern practice (Spring, many frameworks) leans toward unchecked exceptions to reduce boilerplate, wrapping checked exceptions in runtime exceptions.

MUST KNOW

throw vs throws

MUST KNOW

What is the difference between throw and throws?

Typically asked as: Most Frequently Asked

- throw - statement that actually throws an exception object. Used inside method body.
- throws - declaration in method signature that this method may throw the specified exception types. Informs callers that they need to handle these exceptions.

```
// throws - declaration
public void validate(int age) throws InvalidAgeException {
    if (age < 0) {
        // throw - actually throwing
        throw new InvalidAgeException("Age cannot be negative: " + age);
    }
}
```

MUST KNOW

finally and try-with-resources

MUST KNOW

Does finally always execute?

Typically asked as: Tricky

Almost always. The finally block does NOT execute in these rare cases:

- System.exit() is called in try or catch
- The JVM crashes or is killed
- The thread running the code is interrupted/killed
- Infinite loop in try/catch that never completes

Important: If both catch and finally have return statements, the finally return overwrites the catch return.

```
public int tricky() {
    try {
        return 1;
    } finally {
        return 2; // THIS wins - returns 2, not 1 (bad practice!)
    }
}
```

MUST KNOW

What is try-with-resources and why use it?

Typically asked as: Most Frequently Asked

Try-with-resources (Java 7+) automatically closes resources that implement AutoCloseable. It eliminates the verbose finally-block pattern and guarantees proper cleanup even when exceptions occur.

```
// Old way - verbose and error-prone
BufferedReader br = null;
try {
    br = new BufferedReader(new FileReader("file.txt"));
    return br.readLine();
} finally {
    if (br != null) br.close(); // might throw, suppressing original exception
}

// try-with-resources - clean and correct
try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
    return br.readLine();
} // br.close() called automatically, even if exception thrown
```

Multiple resources are closed in reverse order of declaration. Exceptions from close() are suppressed (retrievable via getSuppressed()).

HIGH PRIORITY

Custom Exceptions

HIGH PRIORITY

How do you create a custom exception?

Typically asked as: Most Frequently Asked

```
// Custom checked exception
public class InsufficientFundsException extends Exception {
    private final double deficit;

    public InsufficientFundsException(double deficit) {
        super("Insufficient funds. Deficit: " + deficit);
        this.deficit = deficit;
    }

    public double getDeficit() { return deficit; }
}

// Custom unchecked exception
public class EntityNotFoundException extends RuntimeException {
    public EntityNotFoundException(String entity, Long id) {
        super(entity + " not found with id: " + id);
    }
}
```

Best practices for custom exceptions:

- Extend Exception for recoverable conditions, RuntimeException for programming errors
- Include meaningful context (what failed, relevant IDs, values)
- Provide a constructor that accepts a cause (Throwable cause) for exception chaining
- Name ends with "Exception" by convention

Part 6 - Multithreading Basics

Multithreading questions at the 2-3 year level focus on foundational concepts: how to create threads, synchronization primitives, and common concurrency problems. Deep concurrency (lock-free algorithms, phaser, stamped locks) is typically reserved for senior roles.

MUST KNOW

Thread, Runnable, and Callable

MUST KNOW

What are the ways to create a thread in Java?

Typically asked as: Most Frequently Asked

Three primary approaches:

```
// 1. Extend Thread class (discouraged - wastes inheritance)
class MyThread extends Thread {
    public void run() { System.out.println("Running"); }
}
new MyThread().start();

// 2. Implement Runnable (preferred - separation of concerns)
Runnable task = () -> System.out.println("Running");
new Thread(task).start();

// 3. Implement Callable (returns a result, can throw exceptions)
Callable<Integer> callable = () -> {
    Thread.sleep(1000);
    return 42;
};
ExecutorService executor = Executors.newSingleThreadExecutor();
Future<Integer> future = executor.submit(callable);
int result = future.get(); // blocks until complete
```

MUST KNOW

What is the difference between Runnable and Callable?

Typically asked as: Most Frequently Asked

Feature	Runnable	Callable
Return value	`void`	Generic type `V`
Exceptions	Cannot throw checked exceptions	Can throw checked exceptions
Method	`run()`	`call()`
Used with	Thread, ExecutorService	ExecutorService only

Feature	Runnable	Callable
Result retrieval	Not possible	Via `Future<V>`

HIGH PRIORITY

Executor Framework

HIGH PRIORITY

What is the Executor Framework and why use it?

Typically asked as: Most Frequently Asked

The Executor Framework (java.util.concurrent) manages thread pools, decoupling task submission from thread management. Benefits over raw new Thread():

- Thread reuse (avoids expensive thread creation/destruction)
- Bounded concurrency (control max threads)
- Task queuing when all threads are busy
- Built-in lifecycle management (shutdown, awaitTermination)

```
// Fixed thread pool - bounded concurrency
ExecutorService fixed = Executors.newFixedThreadPool(4);

// Cached thread pool - grows as needed, reuses idle threads
ExecutorService cached = Executors.newCachedThreadPool();

// Scheduled - delayed/periodic execution
ScheduledExecutorService scheduled = Executors.newScheduledThreadPool(2);
scheduled.scheduleAtFixedRate(() -> System.out.println("tick"),
    0, 1, TimeUnit.SECONDS);

// Always shut down when done
fixed.shutdown();
```

MUST KNOW

Synchronization and volatile

MUST KNOW

What is synchronization and why is it needed?

Typically asked as: Most Frequently Asked

Synchronization ensures that only one thread at a time can execute a critical section, preventing race conditions when multiple threads access shared mutable state.

```
public class Counter {
    private int count = 0;

    // Synchronized method - locks on 'this'
    public synchronized void increment() {
        count++; // NOT atomic: read -> increment -> write
    }

    // Synchronized block - finer control
    public void incrementBlock() {
        synchronized (this) {
            count++;
        }
    }
}
```

MUST KNOW

What is the volatile keyword?

Typically asked as: Most Frequently Asked

volatile guarantees visibility: when one thread writes to a volatile variable, all other threads immediately see the new value (bypasses CPU cache). It does NOT guarantee atomicity - volatile int x; x++ is still not thread-safe because x++ involves read + increment + write (three steps).

Use volatile for: flags, status variables, double-checked locking patterns. Use AtomicInteger/synchronized for: counters, compound operations.

```
// Common use: shutdown flag
private volatile boolean running = true;

public void run() {
    while (running) { // guaranteed to see updates from other threads
        doWork();
    }
}

public void stop() {
    running = false; // immediately visible to the thread running run()
}
```

HIGH PRIORITY

Atomic Classes

HIGH PRIORITY

What are Atomic classes and when do you use them?

Typically asked as: Most Frequently Asked

Atomic classes (AtomicInteger, AtomicLong, AtomicReference, etc.) provide lock-free, thread-safe operations on single variables using CAS (Compare-And-Swap) hardware instructions.

```
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet();          // atomic increment
counter.compareAndSet(1, 2);        // CAS: set to 2 only if currently 1
counter.updateAndGet(x -> x * 2);   // atomic compute

// Prefer over synchronized for simple counters - better performance
```

Use Atomic classes instead of synchronized when operations are limited to a single variable. For compound operations involving multiple variables, you still need synchronization.

HIGH PRIORITY

Deadlock

HIGH PRIORITY

What is a deadlock and how do you prevent it?

Typically asked as: Most Frequently Asked

A deadlock occurs when two or more threads are each waiting for the other to release a lock, creating a circular dependency. None can proceed.


```
// Classic deadlock scenario
Object lockA = new Object();
Object lockB = new Object();

// Thread 1: acquires lockA, waits for lockB
new Thread(() -> {
    synchronized (lockA) {
        Thread.sleep(100);
        synchronized (lockB) { /* work */ }
    }
}).start();

// Thread 2: acquires lockB, waits for lockA
new Thread(() -> {
    synchronized (lockB) {
        Thread.sleep(100);
        synchronized (lockA) { /* work */ } // DEADLOCK!
    }
}).start();
```

Prevention strategies:

1. Lock ordering: Always acquire locks in a consistent global order
2. Lock timeout: Use tryLock(timeout) from ReentrantLock
3. Avoid nested locks: Redesign to minimize holding multiple locks
4. Use higher-level concurrency utilities: ConcurrentHashMap, AtomicInteger instead of manual locking

HIGH PRIORITY

Race Condition

HIGH PRIORITY

What is a race condition? Give an example.

Typically asked as: Most Frequently Asked

A race condition occurs when the correctness of a program depends on the relative timing of thread execution. The outcome varies unpredictably between runs.

```
// Classic race condition: check-then-act
public class LazyInit {
    private static Instance instance;

    public static Instance getInstance() {
        if (instance == null) {        // Thread A checks: null
            // Thread B also checks: null (before A completes)
            instance = new Instance(); // Both create instances!
        }
        return instance;
    }
}

// Fix: double-checked locking with volatile
private static volatile Instance instance;
public static Instance getInstance() {
    if (instance == null) {
        synchronized (LazyInit.class) {
            if (instance == null) {
                instance = new Instance();
            }
        }
    }
    return instance;
}
```

Part 7 - JVM & Memory

JVM and memory questions test deeper understanding of how Java runs under the hood. At the 2-3 year level, interviewers focus on the memory model, garbage collection basics, and common memory issues. You are not expected to tune GC parameters, but you should understand the concepts.

MUST KNOW

Heap and Stack

MUST KNOW

What is the difference between heap and stack memory?

Typically asked as: Most Frequently Asked

Feature	Stack	Heap
Stores	Local variables, method frames, references	Objects, instance variables
Lifetime	Method execution scope	Until garbage collected
Thread ownership	Each thread has its own stack	Shared across all threads
Size	Small, fixed (default ~512KB-1MB)	Large, configurable (-Xmx)
Speed	Very fast (LIFO pointer movement)	Slower (allocation + GC)
Error on overflow	StackOverflowError	OutOfMemoryError

```
public void example() {
    int x = 10;            // x stored on STACK
    String name = "hello"; // reference 'name' on STACK
                           // String object "hello" on HEAP (String pool)
    Employee emp = new Employee(); // reference on STACK, object on HEAP
}
```

HIGH PRIORITY

Metaspace

HIGH PRIORITY

What is Metaspace and how does it differ from PermGen?

Typically asked as: Most Frequently Asked

PermGen (removed in Java 8): Fixed-size memory area storing class metadata, String pool, and static variables. Frequently caused java.lang.OutOfMemoryError: PermGen space because its size was bounded and hard to tune.

Metaspace (Java 8+): Stores class metadata in native memory (outside the JVM heap). It grows automatically and is only limited by available system memory (or -XX:MaxMetaspaceSize if explicitly set). This eliminates the PermGen sizing problem.

Note: The String pool was moved to the heap in Java 7, not to Metaspace.

HIGH PRIORITY

Class Loading

HIGH PRIORITY

How does the ClassLoader work in Java?

Typically asked as: Most Frequently Asked

Java uses a delegation hierarchy of ClassLoaders:

1. Bootstrap ClassLoader: Loads core Java classes from rt.jar (java.lang, java.util, etc.)

2. Platform (Extension) ClassLoader: Loads from the ext directory

3. Application ClassLoader: Loads from the classpath (your application code)

Delegation model: When a class needs to be loaded, the request goes UP the hierarchy. Each loader asks its parent first. Only if the parent cannot find the class does the child attempt to load it. This ensures core Java classes are never overridden by user code.

```
// Check which classloader loaded a class
System.out.println(String.class.getClassLoader());     // null (Bootstrap)
System.out.println(MyClass.class.getClassLoader());    // AppClassLoader
```

MUST KNOW

Garbage Collection

MUST KNOW

How does garbage collection work in Java?

Typically asked as: Most Frequently Asked

The JVM automatically reclaims memory occupied by objects that are no longer reachable. An object is eligible for GC when no live thread can reach it through any chain of references.

Generational collection (most JVM implementations):

- Young Generation (Eden + Survivor spaces): New objects are allocated here. Minor GC (fast, frequent) collects short-lived objects.
- Old Generation (Tenured): Objects that survive multiple minor GCs are promoted here. Major GC (slower, less frequent) collects long-lived objects.

GC Roots - objects that are always reachable:

- Local variables in active stack frames

- Static fields of loaded classes
- Active threads
- JNI references

HIGH PRIORITY

What is the difference between minor GC and major GC?

Typically asked as: Follow-up

- Minor GC: Collects Young Generation. Fast (milliseconds) because most young objects are short-lived. Uses copying algorithm - survivors are copied to Survivor space.
- Major GC (Full GC): Collects Old Generation (and sometimes Young). Slower (can pause for seconds). Triggered when Old Gen is full. Compacts memory to reduce fragmentation.

HIGH PRIORITY

Memory Leaks

HIGH PRIORITY

Can Java have memory leaks? Give examples.

Typically asked as: Most Frequently Asked

Yes. A memory leak in Java occurs when objects are no longer needed but still referenced, preventing GC from reclaiming them.

Common causes:

1. Static collections that grow unbounded: static List<Object> cache = new ArrayList<>()

2. Unclosed resources: Database connections, streams, socket connections

3. Event listeners not deregistered: Observer pattern without cleanup

4. Inner class holding reference to outer class: Anonymous inner classes hold implicit reference to the enclosing instance

5. ThreadLocal not removed: Thread pool reuses threads, so ThreadLocal values persist

```
// Memory leak: static map grows forever
public class Cache {
    private static final Map<String, Object> cache = new HashMap<>();
    public static void store(String key, Object value) {
        cache.put(key, value); // never removed!
    }
}

// Fix: use WeakHashMap or implement eviction
private static final Map<String, Object> cache = new WeakHashMap<>();
```

HIGH PRIORITY

OutOfMemoryError and StackOverflowError

HIGH PRIORITY

What causes OutOfMemoryError and StackOverflowError?

Typically asked as: Most Frequently Asked

StackOverflowError: The thread's stack exceeds its size limit. Caused by deep/infinite recursion.

```
// Causes StackOverflowError
public int factorial(int n) {
    return n * factorial(n - 1); // no base case -> infinite recursion
}
```

OutOfMemoryError: The JVM cannot allocate more memory. Common types:

- Java heap space: Heap is full. Caused by memory leaks or insufficient -Xmx.
- Metaspace: Too many classes loaded (common in hot-deploying application servers)
- GC overhead limit exceeded: >98% of time spent in GC with <2% memory recovered

Response strategies:

- StackOverflow -> Fix recursion, add base case, or convert to iterative
- OOM heap -> Increase -Xmx, find memory leak with heap dump, optimize object creation
- OOM Metaspace -> Increase -XX:MaxMetaspaceSize, investigate class loading issues

Part 8 - Java Fundamentals

These foundational concepts are occasionally asked, typically as warmup questions or when an interviewer wants to assess your depth of understanding. They rarely occupy significant interview time but demonstrate solid Java knowledge.

HIGH PRIORITY

JVM, JDK, and JRE

HIGH PRIORITY

What is the difference between JDK, JRE, and JVM?

Typically asked as: Most Frequently Asked

- JVM (Java Virtual Machine): The runtime engine that executes Java bytecode. Responsible for memory management, garbage collection, and platform abstraction. JVM specification allows multiple implementations (HotSpot, GraalVM, OpenJ9).

- JRE (Java Runtime Environment): JVM + standard class libraries + supporting files. Everything needed to RUN a Java application. Does not include development tools.

- JDK (Java Development Kit): JRE + development tools (javac compiler, jdb debugger, jar tool, javadoc). Everything needed to DEVELOP and run Java applications.

Relationship: JDK JRE JVM

Note: Since Java 11, Oracle no longer ships a separate JRE. The JDK is the standard distribution.

HIGH PRIORITY

Bytecode

HIGH PRIORITY

What is Java bytecode and why does it matter?

Typically asked as: Most Frequently Asked

Bytecode is the intermediate representation produced by javac (the Java compiler). It is platform-independent binary code stored in .class files. The JVM interprets or JIT-compiles bytecode into native machine code at runtime.

This two-step process (source -> bytecode -> native) enables Java's "Write Once, Run Anywhere" promise: the same .class file runs on any platform with a compatible JVM.

```
// Source code: Hello.java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello");
    }
}
// javac Hello.java -> Hello.class (bytecode)
// java Hello -> JVM loads Hello.class, JIT-compiles, executes
```

HIGH PRIORITY

Object Class

HIGH PRIORITY

What are the important methods of the Object class?

Typically asked as: Most Frequently Asked

Every Java class implicitly extends Object. Key methods:

Method	Purpose	When to Override
`equals(Object)`	Logical equality	When objects need value-based comparison
`hashCode()`	Hash for collections	Always override with equals()
`toString()`	String representation	For debugging/logging
`clone()`	Shallow copy	Rarely (use copy constructors instead)
`finalize()`	Cleanup before GC	Never (deprecated, use try-with-resources)
`getClass()`	Runtime class info	Never override (final)
`wait()/notify()/notifyAll()`	Thread communication	Rarely (prefer java.util.concurrent)

MEDIUM PRIORITY

Keywords and Access Modifiers

MEDIUM PRIORITY

What are the access modifiers in Java and their scope?

Typically asked as: Most Frequently Asked

Modifier	Class	Package	Subclass	World
`private`				
default (no modifier)				
`protected`				
`public`				

HIGH PRIORITY

What does the `final` keyword mean in different contexts?

Typically asked as: Most Frequently Asked

- final variable: Cannot be reassigned after initialization. For object references, the reference is fixed but the object's state can still be modified.
- final method: Cannot be overridden by subclasses.
- final class: Cannot be extended (no subclasses). Examples: String, Integer, System.

```
final int x = 10;
// x = 20; // Compile error

final List<String> list = new ArrayList<>();
list.add("hello"); // OK - modifying object state
// list = new ArrayList<>(); // Compile error - reassigning reference

final class Immutable { /* cannot be extended */ }
```

HIGH PRIORITY

What is the difference between `final`, `finally`, and `finalize()`?

Typically asked as: Tricky

- final - keyword for constants, preventing override/extension (see above)
- finally - block that executes after try/catch regardless of exception
- finalize() - deprecated Object method called by GC before reclaiming an object. Never use it; prefer AutoCloseable and try-with-resources.

Part 9 - Less Frequently Asked Topics

This appendix covers topics that appear occasionally in interviews. They are worth reviewing if you have extra preparation time, but prioritize Parts 1-6 first.

GOOD TO KNOW

Serialization

GOOD TO KNOW

What is serialization in Java?

Typically asked as: Most Frequently Asked (when asked)

Serialization converts an object into a byte stream for storage or transmission. The object must implement java.io.Serializable (a marker interface).

- transient keyword excludes fields from serialization
- serialVersionUID should be explicitly defined to avoid InvalidClassException during deserialization when the class structure changes

```
public class User implements Serializable {
    private static final long serialVersionUID = 1L;
    private String name;
    private transient String password; // excluded from serialization
}
```

MEDIUM PRIORITY

Immutable Class Design

MEDIUM PRIORITY

How do you create an immutable class in Java?

Typically asked as: Most Frequently Asked (when asked)

Rules for immutability:

1. Declare class as final (prevent subclass modification)
2. Make all fields private and final
3. No setter methods
4. Deep-copy mutable objects in constructor and getters
5. Don't expose mutable internal state

```
public final class Money {
    private final BigDecimal amount;
    private final Currency currency;

    public Money(BigDecimal amount, Currency currency) {
        this.amount = amount;
        this.currency = Currency.getInstance(currency.getCurrencyCode());
    }

    public BigDecimal getAmount() { return amount; } // BigDecimal is immutable
    public Currency getCurrency() {
        return Currency.getInstance(currency.getCurrencyCode()); // defensive copy
    }
}
```

MEDIUM PRIORITY

Generics

MEDIUM PRIORITY

What is type erasure in Java generics?

Typically asked as: Most Frequently Asked (when asked)

Type erasure is the process by which the compiler removes all generic type information at compile time, replacing type parameters with their bounds (or Object if unbounded). This means generic types do not exist at runtime.

Consequences:

- Cannot do new T() or new T[]
- Cannot do instanceof List<String> (only instanceof List)
- Cannot have List<int> (primitives not allowed, use List<Integer>)
- A List<String> and List<Integer> are the same List class at runtime

```
// At compile time:
List<String> strings = new ArrayList<>();
strings.add("hello");
String s = strings.get(0);

// After type erasure (what the JVM sees):
List strings = new ArrayList();
strings.add("hello");
String s = (String) strings.get(0); // cast inserted by compiler
```

GOOD TO KNOW

Design Patterns in Interviews

GOOD TO KNOW

Explain the Singleton pattern and its thread-safe implementations.

Typically asked as: Most Frequently Asked (when asked)

```
// Approach 1: Enum Singleton (recommended by Effective Java)
public enum Singleton {
    INSTANCE;
    public void doSomething() { }
}

// Approach 2: Bill Pugh (static inner class)
public class Singleton {
    private Singleton() {}
    private static class Holder {
        private static final Singleton INSTANCE = new Singleton();
    }
    public static Singleton getInstance() {
        return Holder.INSTANCE;
    }
}
```

The Enum approach is simplest and handles serialization, reflection attacks, and thread safety automatically.

Let's Connect

I hope this guide makes your next interview feel a little less daunting. If it helped, or if you spot something worth improving, I would genuinely love to hear from you. Feel free to reach out or follow along on the platforms below.

LinkedIn

Kolisetty Surya Mahesh

GitHub

SuryaMahesh789 (Kolisetty Surya Mahesh)

All the best for your interviews.